THE BITCOIN CORPORATION

Contracts Implementation

Smart Contract Development Guidelines

Document Version: 1.0 | Date: November 2025 | The Bitcoin Corporation LTD

Bitcoin Contracts Repository Setup

📁 Repository: bitcoin-corp/bitcoin-contracts

This repository should contain all the smart contract templates for the Bitcoin Corporation multi-token ecosystem.

🏗️ Suggested Structure

```
bitcoin-corp/bitcoin-contracts/
├── README.md # Overview and documentation
├── contracts/
│ ├── core/
│ │ ├── MasterCreator.sol # Base contract all inherit from
│ │ ├── TokenFactory.sol # Creates new app tokens
│ │ └── AtomicSwap.sol # Token exchange mechanism
│ ├── templates/
│ │ ├── DeveloperContract.sol # For code contributors
│ │ ├── CreatorContract.sol # For content creators
│ │ ├── DesignerContract.sol # For UI/UX contributors
│ │ └── TesterContract.sol # For QA/testing
│ ├── tokens/
│ │ ├── bCorp.sol # Master corporation token
│ │ ├── bOS.sol # Operating system token
```

```
│ │ └── AppToken.sol # Template for app tokens
│ └── governance/
│ ├── DAO.sol # Decentralized governance
│ └── Treasury.sol # Revenue distribution
├── scripts/
│ ├── deploy.js # Deployment scripts
│ ├── distribute.js # Token distribution
│ └── githubIntegration.js # GitHub Actions hooks
├── test/
│ └── *.test.js # Contract tests
└── package.json # Dependencies
```

📝 Core Contracts to Implement

1. MasterCreator.sol

```solidity
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;
/
* @title MasterCreator
* @dev Base contract for all Bitcoin Corp creator contracts
*/
contract MasterCreator {
struct Creator {
address wallet;
string githubUsername;
mapping(string => uint256) tokenBalances;
uint256 totalContributions;
uint256 joinedAt;
}
```

```
mapping(address => Creator) public creators;

mapping(string => address) public tokenContracts;

event TokensEarned(address creator, string token, uint256 amount);

event CreatorRegistered(address creator, string githubUsername);

}
```

2. DeveloperContract.sol

```
solidity

// SPDX-License-Identifier: MIT

pragma solidity ^0.8.0;

import "./core/MasterCreator.sol";

/

* @title DeveloperContract

* @dev Handles developer contributions and token rewards

*/

contract DeveloperContract is MasterCreator {

struct Contribution {

string repository;

string pullRequestId;

uint256 linesAdded;

uint256 linesRemoved;

uint256 filesChanged;

uint256 complexity;

uint256 reward;

uint256 timestamp;

}

mapping(address => Contribution[]) public contributions;

// Base rates per repository

mapping(string => uint256) public repoRates;

constructor() {
```

```
// Set initial rates
repoRates["bitcoin-os"] = 10;
repoRates["bitcoin-music"] = 8;
repoRates["bitcoin-writer"] = 8;
repoRates["bitcoin-email"] = 7;
// ... etc
}
function submitContribution(
address developer,
string memory repo,
string memory prId,
uint256 linesAdded,
uint256 linesRemoved,
uint256 complexity
) external {
uint256 reward = calculateReward(repo, linesAdded, complexity);
contributions[developer].push(Contribution({
repository: repo,
pullRequestId: prId,
linesAdded: linesAdded,
linesRemoved: linesRemoved,
filesChanged: 0,
complexity: complexity,
reward: reward,
timestamp: block.timestamp
}));
// Issue tokens
_issueTokens(developer, repo, reward);
}
function calculateReward(
string memory repo,
```

```
uint256 lines,

uint256 complexity

) public view returns (uint256) {

return lines complexity repoRates[repo];

}

}
```

3. AtomicSwap.sol

```
solidity

// SPDX-License-Identifier: MIT

pragma solidity ^0.8.0;

/

* @title AtomicSwap

* @dev Enables token swapping between different app tokens

*/

contract AtomicSwap {

struct SwapRate {

uint256 rate; // Rate * 100 for precision

uint256 lastUpdated;

}

// From token => To token => Rate

mapping(string => mapping(string => SwapRate)) public rates;

event SwapExecuted(

address indexed user,

string fromToken,

string toToken,

uint256 amountIn,

uint256 amountOut

);

constructor() {
```

```
// Set initial exchange rates

rates["bCorp"]["bOS"] = SwapRate(200, block.timestamp); // 1 bCorp = 2 bOS

rates["bOS"]["bMusic"] = SwapRate(150, block.timestamp); // 1 bOS = 1.5 bMusic

rates["bOS"]["bWriter"] = SwapRate(150, block.timestamp); // 1 bOS = 1.5 bWriter

rates["bMusic"]["bWriter"] = SwapRate(100, block.timestamp); // 1 bMusic = 1
bWriter

}

function swap(

string memory fromToken,

string memory toToken,

uint256 amount

) external {

require(rates[fromToken][toToken].rate > 0, "Pair not supported");

uint256 outputAmount = (amount * rates[fromToken][toToken].rate) / 100;

// Execute swap logic here

emit SwapExecuted(msg.sender, fromToken, toToken, amount, outputAmount);

}

}
```

🔧 GitHub Integration Script

github-action.yml

```
yaml

name: Token Distribution

on:

pull_request:

types: [closed]

branches: [main]

jobs:

distribute:

if: github.event.pull_request.merged == true
```

```
runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v2

- name: Calculate Contribution

id: calculate

run: |

echo "LINES_ADDED=$(git diff --numstat ${{ github.event.pull_request.base.sha
}}..${{ github.event.pull_request.head.sha }} | awk '{sum+=$1} END {print sum}')"
>> $GITHUB_OUTPUT

echo "LINES_REMOVED=$(git diff --numstat ${{ github.event.pull_request.base.sha
}}..${{ github.event.pull_request.head.sha }} | awk '{sum+=$2} END {print sum}')"
>> $GITHUB_OUTPUT

- name: Submit to Smart Contract

run: |

node scripts/submitContribution.js \

--developer "${{ github.event.pull_request.user.login }}" \

--repo "${{ github.repository }}" \

--pr "${{ github.event.pull_request.number }}" \

--added "${{ steps.calculate.outputs.LINES_ADDED }}" \

--removed "${{ steps.calculate.outputs.LINES_REMOVED }}"
```

📊 Token Distribution Matrix

Repository Token Base Rate Complexity Multiplier |

----------|

bitcoin-os $bOS 10 1-5x |

bitcoin-music $bMusic 8 1-3x |

bitcoin-writer $bWriter 8 1-3x |

bitcoin-email $bEmail 7 1-3x |

bitcoin-wallet $bWallet 12 1-5x (security critical) |

bitcoin-contracts $bCorp 15 1-5x (infrastructure) |

🚀 Deployment Steps

1. **Clone and setup**

```bash
git clone https://github.com/bitcoin-corp/bitcoin-contracts
cd bitcoin-contracts
npm install
```

2. **Deploy contracts**

```bash
npm run deploy:mainnet
```

3. **Verify contracts**

```bash
npm run verify
```

4. **Set up GitHub Actions**

```bash
# Add secrets to each repository
# - CONTRACT_ADDRESS
# - PRIVATE_KEY
# - RPC_URL
```

🎯 Next Steps

1. **Implement contracts** in bitcoin-contracts repo

2. **Deploy to testnet** first

3. **Integrate GitHub Actions** in each app repo

4. **Create dashboard** for tracking contributions

5. **Launch token distribution** program

This creates a complete automated token economy where:

- Every PR = automatic token calculation
- Every merge = instant token distribution
- Every developer = transparent earnings
- Every app = its own economy

The contracts repository becomes the backbone of the entire Bitcoin Corporation token ecosystem!